AgriKart – Complete Docker + AWS EC2 Deployment Guide

Step-by-step explanation of the deployment process, commands used, architecture, troubleshooting, Git/GitHub, Docker Hub, and database setup.

Project stack: Flask + MySQL + HTML/CSS/JavaScript + Nginx + Docker + Docker Compose + Docker Hub + AWS EC2 + Git/GitHub

Important: This guide documents the deployment work actually performed in the project. It does not claim that GitHub Actions CI/CD was already configured.

1. Project Objective

The objective was to take an existing Flask/MySQL AgriKart e-commerce application and make it deployment-ready. The application was separated into frontend, backend, and MySQL containers, with Nginx serving the frontend and proxying API requests to Flask. Docker images were published to Docker Hub and the application was deployed on an AWS EC2 instance using Docker Compose.

Final flow: Browser → Nginx/frontend container → Flask backend container → MySQL container

2. Application Structure and Components

The deployment used three main services:

Service	Purpose	Image
frontend	Nginx serves website files and proxies /api requests	pramodsadekar1718/agrikart:frontend
backend	Flask API and application logic	pramodsadekar1718/agrikart:backend
db	MySQL database	pramodsadekar1718/agrikart:mysql

3. Dockerfiles and Containerization

The project used separate Docker images for the frontend and backend. MySQL was also packaged/published as a Docker image.

Typical image build commands used from the project directory:

```
docker build -f Dockerfile.frontend -t pramodsadekar1718/agrikart:frontend .
docker build -t pramodsadekar1718/agrikart:backend .
```

Check local images:

```
docker images
```

4. Docker Hub

The three images were pushed to Docker Hub so the EC2 server could pull the same images without building the application there.

```
docker login
docker push pramodsadekar1718/agrikart:frontend
docker push pramodsadekar1718/agrikart:backend
docker push pramodsadekar1718/agrikart:mysql
```

Images used on EC2:

```
pramodsadekar1718/agrikart:frontend
pramodsadekar1718/agrikart:backend
```

```
pramodsadekar1718/agrikart:mysql
```

5. Docker Compose

Docker Compose was used to run the three services together. The important production settings were:

```
services:
  db:
    image: pramodsadekar1718/agrikart:mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: agrikart_local_root
      MYSQL_DATABASE: agrikart_db
      MYSQL_USER: agrikart_app
      MYSQL_PASSWORD: agrikart_local_password
    volumes:
      - mysql_data:/var/lib/mysql

  backend:
    image: pramodsadekar1718/agrikart:backend
    restart: unless-stopped
    environment:
      FLASK_HOST: 0.0.0.0
      FLASK_DEBUG: "False"
      SECRET_KEY: change-this-secret
      DB_HOST: db
      DB_PORT: 3306
      DB_NAME: agrikart_db
      DB_USER: agrikart_app
      DB_PASSWORD: agrikart_local_password
    volumes:
      - product_uploads:/app/uploads
    depends_on:
      db:
        condition: service_healthy

  frontend:
    image: pramodsadekar1718/agrikart:frontend
    restart: unless-stopped
    volumes:
      - product_uploads:/usr/share/nginx/html/uploads:ro
    ports:
      - "8081:80"
    depends_on:
      - backend

volumes:
  mysql_data:
  product_uploads:
```

6. Why DB_HOST=db Was Used

Inside a Docker Compose network, services can communicate using their service names. Therefore the Flask backend connects to MySQL using **db**, not localhost. localhost inside the backend container means the backend container itself.

```
DB_HOST=db
DB_PORT=3306
```

7. AWS EC2 Creation

An Amazon Linux 2023 EC2 instance was used. The instance type used was t3.micro. The application was exposed through TCP port 8081.

Security Group inbound rules used: SSH 22, HTTP 80, and TCP 8081. For real production security, SSH should be restricted to your IP and HTTPS should be configured.

8. Connect to EC2

```
ssh -i /path/to/your-key.pem ec2-user@YOUR_EC2_PUBLIC_IP
```

Example format:

```
ssh -i /mnt/c/Users/Pramo/Downloads/YOUR-KEY.pem ec2-user@3.111.38.31
```

9. Install Docker on Amazon Linux

```
sudo dnf update -y
sudo dnf install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```

Reconnect to EC2 after adding the user to the Docker group, then verify:

```
docker --version
docker ps
```

10. Install Docker Compose

```
docker compose version
```

Docker Compose was used through the Docker CLI as **docker compose**.

11. Create the Application Directory

```
mkdir -p /home/ec2-user/agrikart
cd /home/ec2-user/agrikart
pwd
```

12. Get Project Files Using Git/GitHub

Git was installed on EC2 and the existing directory was connected to the GitHub repository.

```
sudo dnf install git -y
cd /home/ec2-user/agrikart
git init
git branch -M main
git remote add origin https://github.com/Pramod-sadekar18/agrikart.git
git fetch origin
git checkout origin/main -- .
git remote -v
```

The production Docker Compose file was backed up before bringing GitHub files into the directory:

```
cp docker-compose.yml docker-compose.ec2.backup.yml
```

This backup was important because the GitHub Compose file and the EC2 production Compose were not identical.

13. Database Schema Problem

Initially the MySQL container was healthy, but the database had no application tables. This caused Flask API requests such as /api/products and /api/register to return HTTP 500 errors.

The database was checked with:

```
docker compose exec db mysql -uagrikart_app -pagrikart_local_password agrikart_db -e "SHOW TABLES;
```

The result showed no tables. The root cause was that the production Compose file did not automatically mount the schema into the already-created MySQL data volume.

14. Import the Database Schema

The schema file used was **database/schema.sql**. It was imported directly into the running MySQL container:

```
docker compose exec -T db mysql -uagrikart_app -pagrikart_local_password agrikart_db < database/schema.sql
```

A standard MySQL warning about using a password on the command line was displayed; the import completed successfully.

15. Start the Application

```
cd /home/ec2-user/agrikart
docker compose up -d
```

Check service status:

```
docker compose ps
```

Expected services: db, backend, frontend.

16. Check Logs

```
docker compose logs db
docker compose logs backend
docker compose logs frontend
```

For live logs:

```
docker compose logs -f backend
```

17. Verify MySQL Health

```
docker compose ps
docker compose exec db mysqladmin ping -h 127.0.0.1 -uroot -p"agrikart_local_root" --silent
```

The Compose healthcheck also checks whether MySQL is ready before the backend starts.

18. Verify Flask Backend

The Flask application was configured to listen on all interfaces:

```
FLASK_HOST=0.0.0.0
FLASK_DEBUG=False
```

This is important because a server bound only to 127.0.0.1 would not be reachable from the Nginx container.

19. Nginx Reverse Proxy

Nginx serves the static frontend and forwards API requests beginning with /api to the backend container. This allows the browser to use the same public host while Nginx handles routing to Flask.

Conceptual flow:

```
Browser
   |
   | http://EC2-IP:8081
   v
Nginx / frontend
   |
   | /api/*
   v
Flask backend
   |
   v
MySQL
```

20. Verify the Public Port

```
sudo ss -lntp | grep 8081
curl -I http://localhost:8081
```

Port 8081 was mapped from EC2 to Nginx container port 80:

```
ports:
  - "8081:80
```

21. Test the API

The final successful test was:

```
curl http://localhost:8081/api/products
```

The response returned JSON with **success=true** and a product count of **34**, confirming that Nginx, Flask, Docker networking, and MySQL were working together.

22. Open the Website

```
http://EC2_PUBLIC_IP:8081
```

At the time of the deployment, the public address was http://3.111.38.31:8081. The public IPv4 address can change after stopping and starting an EC2 instance unless an Elastic IP is used.

23. Docker Volumes and Data Persistence

Two named volumes were used:

```
mysql_data
product_uploads
```

mysql_data stores MySQL database files. product_uploads stores uploaded product files and is shared between backend and Nginx. Do not use **docker compose down -v** on the production system because it can remove named volumes and therefore database data.

24. Restart After EC2 Stop/Start

Stopping an EC2 instance for a few days does not delete the EBS disk or Docker volumes. After starting the instance again, connect by SSH and check:

```
cd /home/ec2-user/agrikart
docker compose ps
curl http://localhost:8081/api/products
```

If the containers are not running, start them:

```
docker compose up -d
```

25. Git Workflow Used

Git/GitHub was used to keep the project source code version-controlled and available on the EC2 server.

```
git status
git remote -v
git fetch origin
git checkout origin/main -- .
```

A future deployment workflow can be automated with GitHub Actions, but that CI/CD automation was not part of the deployment completed so far.

26. Troubleshooting Performed

Problem: API returned HTTP 500.
Investigation: Checked Nginx logs, backend logs, MySQL health, and database tables.
Finding: MySQL was healthy but the application database had no tables.
Fix: Imported database/schema.sql into agrikart_db.
Verification: /api/products returned 34 products with success=true.

27. Useful Daily Commands

```
# Check containers
docker compose ps

# Start
docker compose up -d

# Stop containers
docker compose stop

# Restart
docker compose restart

# View logs
docker compose logs -f

# Backend logs
docker compose logs -f backend

# Pull latest Docker Hub images
docker compose pull

# Check Docker images
docker images

# Check volumes
docker volume ls

# Check disk
df -h

# Check listening port
sudo ss -lntp | grep 8081
```

28. Important Production Improvements

The deployment is production-oriented, but it is not fully production-hardened yet. Before treating it as a public production system, address the following:

1. Move database passwords and SECRET_KEY into secure environment configuration rather than keeping them directly in Compose.
2. Replace the placeholder SECRET_KEY.
3. Restrict SSH port 22 to your own IP where possible.
4. Add HTTPS/TLS, preferably through Nginx/Let's Encrypt or an AWS load balancer.
5. Set up database backups.
6. Add monitoring/logging.
7. Consider an Elastic IP or domain name so the public address does not change after EC2 stop/start.

29. Interview Explanation

Short answer: “I worked on deploying an existing Flask/MySQL AgriKart application on AWS EC2 using Docker. I created separate Docker images for the frontend, backend, and MySQL, pushed them to Docker Hub, configured Docker Compose with service networking and persistent volumes, configured Nginx to serve the frontend and proxy API requests, and deployed the stack on EC2. During deployment the API initially returned 500 because the MySQL database had no tables, so I investigated the containers and logs, imported the schema.sql file, and verified the API successfully returned product data.”

30. End-to-End Command Checklist

```
# LOCAL / BUILD
docker build -f Dockerfile.frontend -t pramodsadekar1718/agrikart:frontend .
docker build -t pramodsadekar1718/agrikart:backend .
docker images
```

```
# DOCKER HUB
docker login
docker push pramodsadekar1718/agrikart:frontend
docker push pramodsadekar1718/agrikart:backend
docker push pramodsadekar1718/agrikart:mysql

# EC2
sudo dnf update -y
sudo dnf install docker git -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user

mkdir -p /home/ec2-user/agrikart
cd /home/ec2-user/agrikart

# GIT
git init
git branch -M main
git remote add origin https://github.com/Pramod-sadekar18/agrikart.git
git fetch origin
git checkout origin/main -- .

# DEPLOY
docker compose pull
docker compose up -d
docker compose ps

# DATABASE
docker compose exec db mysql -uagrikart_app -pagrikart_local_password agrikart_db -e "SHOW TABLES;"
docker compose exec -T db mysql -uagrikart_app -pagrikart_local_password agrikart_db < database/schema.sql

# VERIFY
docker compose ps
curl http://localhost:8081/api/products
curl -I http://localhost:8081
sudo ss -lntp | grep 8081

# WEBSITE
http://EC2_PUBLIC_IP:8081
```

Note on security: The passwords shown above are the local/deployment values used in the project context. For a real public deployment, rotate exposed credentials and store secrets securely. Do not commit real secrets to GitHub.